UTS

PEMBELAJARAN MESIN

NAMA : Arya Shahryar Asis

NIM : 202431116

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 18

ASISTEN : 1. Muhammad Caisar Gemilang Pratama

2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI .....
BAB I 3
PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II 4
LANDASAN TEORI 4
2.1 Pembelajaran Mesin dan Klasifikasi Terawasi 4
2.2 Prediksi Hasil Ronde pada Counter-Strike 4
2.3 Exploratory Data Analysis dan Prapemrosesan Data 4
2.4 Algoritma Naive Bayes 5
2.5 Algoritma K-Nearest Neighbors (KNN) 6
2.6 Validasi Silang dan Penyetelan Hiperparameter 6
2.7 Metrik Evaluasi Klasifikasi 7
BAB III 8
HASIL 8
3.1 Deskripsi Dataset dan Lingkungan Pengerjaan 8
Gambar 3.1 8
3.2 Exploratory Data Analysis (EDA) 9
Gambar 3.2 9
3.3 Distribusi Kelas Target 10
Gambar 3.3 10
3.4 Hubungan Fitur dengan Pemenang Ronde 11
3.5 GAMBAR CODINGAN 12
..... 13
3.6 Penjelasan Program 15
BAB IV 18
PENUTUP 18
4.1 Kesimpulan 18
DAFTAR PUSTAKA 19

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

- Bagaimana karakteristik dataset *Counter Strike 2* berdasarkan hasil *Exploratory Data Analysis* (EDA), meliputi struktur data, nilai hilang, data duplikat, distribusi kelas target, dan hubungan antarfitur?
- Tahapan prapemrosesan apa yang diperlukan agar data siap digunakan oleh algoritma *Naive Bayes* dan *K-Nearest Neighbors* (KNN)?
- Bagaimana kinerja algoritma *Naive Bayes* dalam memprediksi pemenang ronde (CT atau T) yang diukur dengan akurasi, *precision*, *recall*, *F1-score*, dan AUC?

1.2 Tujuan Masalah

- Memahami karakteristik dataset *Counter Strike 2* melalui proses EDA, termasuk pemeriksaan kualitas data dan hubungan antarfitur dengan pemenang ronde.
- Menerapkan prapemrosesan data berupa penghapusan duplikat, analisis *outlier*, *encoding* data kategorikal, pembagian data secara berstrata, dan standarisasi fitur.

1.3 Manfaat Masalah

- Hasil UTS Praktikum menunjukkan bahwa kondisi sumber daya tim, seperti kesehatan, pelindung, uang, dan status bom, dapat dimanfaatkan untuk memprediksi pemenang ronde pada gim Counter-Strike. Pengetahuan ini dapat menjadi dasar bagi analisis performa tim dan pengembangan sistem prediksi pertandingan *esports* yang lebih lanjut.

BAB II
LANDASAN TEORI

2.1 Pembelajaran Mesin dan Klasifikasi Terawasi

Pembelajaran mesin (*machine learning*) merupakan cabang kecerdasan buatan yang memungkinkan komputer mempelajari pola dari data sehingga dapat membuat keputusan atau prediksi tanpa harus diprogram dengan aturan eksplisit untuk setiap kondisi. Pada pembelajaran terawasi (*supervised learning*), model dilatih menggunakan data yang telah memiliki label sehingga mampu memetakan sekumpulan fitur masukan ke label keluaran. Apabila label berupa kategori diskret, tugas tersebut dinamakan klasifikasi, sedangkan apabila berupa nilai kontinu dinamakan regresi (Rainio et al., 2024).

Praktikum UTS ini merupakan kasus klasifikasi biner, karena variabel target `round_winner` hanya memiliki dua kelas, yaitu CT (*Counter-Terrorist*) dan T (*Terrorist*). Tahapan kerja yang diikuti mencakup pemahaman data melalui *Exploratory Data Analysis* (EDA), prapemrosesan data, pelatihan model, dan evaluasi hasil prediksi menggunakan metrik yang sesuai. Dua algoritma yang digunakan adalah *Naive Bayes* dan *K-Nearest Neighbors* (KNN).

2.2 Prediksi Hasil Ronde pada Counter-Strike

Counter-Strike adalah gim tembak-menembak taktis berbasis ronde. Dua tim, yaitu CT dan T, saling berhadapan pada satu peta (*map*) dengan tujuan yang berbeda: tim T berupaya menanam bom, sedangkan tim CT berupaya mencegahnya atau menjinakkan bom yang telah ditanam. Server gim mencatat kondisi permainan secara rinci, seperti sisa waktu, skor, total kesehatan dan pelindung (*armor*) tiap tim, uang tim, serta status bom. Data tersebut dapat dipotret pada suatu momen (*snapshot*) dan digunakan sebagai fitur untuk memprediksi tim yang memenangkan ronde.

Prediksi pemenang ronde pada Counter-Strike telah diteliti dengan berbagai pendekatan. Sinap (2023) membandingkan beberapa algoritma klasifikasi untuk memprediksi hasil ronde pada Counter-Strike: Global Offensive. Xenopoulos et al. (2021) membangun model probabilitas kemenangan dengan memanfaatkan fitur seperti skor tim, perlengkapan, dan uang tim, sedangkan Hasegawa dan Hirota (2024) memprediksi tim pemenang dari informasi *frame*, antara lain poin kesehatan dan perlengkapan pemain, menggunakan XGBoost. Kesamaan dari penelitian-penelitian tersebut adalah bahwa kondisi sumber daya tim pada saat tertentu memuat informasi yang cukup untuk memprediksi hasil ronde.

2.3 Exploratory Data Analysis dan Prapemrosesan Data

EDA dilakukan untuk memahami struktur dan karakteristik data sebelum pemodelan, meliputi pemeriksaan ukuran data, tipe data, nilai hilang (*missing value*), data duplikat, ringkasan statistik,

distribusi kelas target, serta hubungan antarfitur melalui matriks korelasi. Temuan EDA menentukan langkah prapemrosesan yang diperlukan. Beberapa langkah prapemrosesan yang digunakan pada praktikum ini diuraikan berikut.

- **Penghapusan data duplikat.** Baris yang identik dihapus agar data yang sama tidak muncul pada data latih sekaligus data uji, karena hal tersebut membuat akurasi tampak lebih baik daripada yang sebenarnya.
- **Deteksi outlier dengan metode IQR.** Suatu nilai dianggap pencilan apabila berada di luar rentang $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$, dengan $IQR = Q_3 - Q_1$. Pencilan perlu dianalisis terlebih dahulu sebelum diputuskan untuk dihapus atau dipertahankan, karena belum tentu merupakan kesalahan data.
- **Encoding data kategorikal.** Algoritma pembelajaran mesin memerlukan masukan numerik, sehingga fitur kategorikal seperti `map` diubah menjadi kolom biner dengan *one-hot encoding*, fitur boolean diubah menjadi 0 dan 1, dan label target diubah menjadi bilangan menggunakan *label encoding*.
- **Standarisasi fitur (z-score).** Setiap fitur ditransformasi menjadi $z = (x - \mu) / \sigma$ sehingga memiliki rata-rata 0 dan simpangan baku 1. Langkah ini penting bagi algoritma berbasis jarak seperti KNN, karena fitur berskala besar akan mendominasi perhitungan jarak (Pagan et al., 2023). Dampak berbagai teknik penskalaan terhadap kinerja algoritma klasifikasi juga telah dikaji secara luas oleh Pinheiro et al. (2025).
- *Pembagian data secara berstrata (*stratified split*). Data dibagi menjadi data latih dan data uji dengan proporsi kelas yang dijaga sama.* Scaler *hanya dilatih (*fit*) pada data latih lalu diterapkan pada data uji agar tidak terjadi kebocoran informasi (*data leakage*).

2.4 Algoritma Naive Bayes

Naive Bayes merupakan metode klasifikasi probabilistik yang berlandaskan Teorema Bayes. Metode ini banyak digunakan karena sederhana, cepat dilatih, dan mudah diimplementasikan (Liu et al., 2024). Untuk vektor fitur $X = (x_1, x_2, \ldots, x_n)$ dan kelas C, probabilitas posterior kelas dihitung dengan:

$$P(C \mid X) = P(X \mid C) \cdot P(C) / P(X)$$

Kata *naive* (naif) merujuk pada asumsi independensi bersyarat, yaitu setiap fitur dianggap saling bebas apabila kelasnya diketahui, sehingga likelihood dapat difaktorkan menjadi:

$$P(X \mid C) = P(x_1 \mid C) \cdot P(x_2 \mid C) \cdot \ldots \cdot P(x_n \mid C)$$

Kelas yang diprediksi adalah kelas dengan probabilitas posterior terbesar. Pada fitur numerik kontinu digunakan varian *Gaussian Naive Bayes*, yang mengasumsikan setiap fitur pada tiap kelas berdistribusi normal dengan rata-rata μ_c dan varians σ^2_c:

$$P(x_i \mid C) = (1 / \sqrt{(2\pi\ \sigma^2_c)}) \cdot \exp(\ -(x_i - \mu_c)^2 / (2\ \sigma^2_c)\ )$$

Kelebihan *Naive Bayes* adalah komputasinya ringan dan tetap bekerja dengan data latih yang relatif sedikit. Kelemahan utamanya terletak pada asumsi independensi yang pada data nyata sering tidak terpenuhi karena fitur-fitur saling berkorelasi, sehingga estimasi probabilitasnya dapat menjadi kurang akurat (Liu et al., 2024). Selain itu, varian Gaussian kurang sesuai apabila distribusi fitur sangat miring (*skewed*).

2.5 Algoritma K-Nearest Neighbors (KNN)

KNN adalah metode pembelajaran terawasi yang bersifat non-parametrik dan berbasis instans (*instance-based*). KNN tergolong *lazy learner* karena tidak membangun model eksplisit pada tahap pelatihan, melainkan menyimpan seluruh data latih dan baru melakukan perhitungan saat ada data baru yang harus diprediksi (Halder et al., 2024). Langkah kerjanya adalah (1) menentukan nilai k, (2) menghitung jarak data baru terhadap seluruh data latih, (3) memilih k data dengan jarak terkecil, dan (4) menentukan kelas melalui suara mayoritas dari k tetangga tersebut.

Dua ukuran jarak yang umum digunakan untuk dua titik x dan y dengan n fitur adalah jarak Euclidean dan jarak Manhattan:

$$d_{\text{Euclidean}}(x, y) = \sqrt{(\ \Sigma\ (x_i - y_i)^2\ )} \qquad d_{\text{Manhattan}}(x, y) = \Sigma\ |x_i - y_i|$$

Pemilihan ukuran jarak dapat memengaruhi akurasi secara signifikan bergantung pada karakteristik data. Nayak et al. (2022) menguji berbagai metrik jarak dan rentang nilai k pada KNN dan menemukan bahwa performa tiap metrik berbeda pada dataset yang dikaji. Nilai k juga merupakan hiperparameter yang krusial. Nilai k yang kecil membuat model sensitif terhadap *noise* dan cenderung *overfitting*, sedangkan nilai k yang besar menghaluskan batas keputusan dan dapat menyebabkan *underfitting* (Halder et al., 2024). Zhang (2022) menegaskan bahwa penentuan k, pemilihan tetangga, pencarian tetangga terdekat, dan aturan klasifikasi masih menjadi tantangan utama pada KNN.

Variasi KNN berbobot (*weighted KNN*) memberikan bobot yang lebih besar kepada tetangga yang lebih dekat dan bobot yang lebih kecil kepada tetangga yang lebih jauh, berbeda dengan KNN standar yang memperlakukan seluruh tetangga secara setara (Halder et al., 2024). Kelemahan KNN antara lain biaya komputasi yang tinggi pada data besar, penurunan kinerja pada data berdimensi tinggi (*curse of dimensionality*), serta sensitivitas terhadap *noise* dan *outlier* (Halder et al., 2024).

2.6 Validasi Silang dan Penyetelan Hiperparameter

Validasi silang *k-fold* membagi data latih menjadi k lipatan; model dilatih pada k−1 lipatan dan divalidasi pada satu lipatan sisanya, dan proses diulang sebanyak k kali sehingga setiap lipatan menjadi data validasi satu kali. Rata-rata dan simpangan baku skor dari seluruh lipatan menggambarkan kinerja sekaligus kestabilan model. Varian *stratified* menjaga proporsi kelas pada

setiap lipatan. Penyetelan hiperparameter dengan *Grid Search* mencoba seluruh kombinasi nilai hiperparameter yang ditentukan, yaitu jumlah tetangga, skema bobot, dan metrik jarak pada KNN, lalu memilih kombinasi dengan skor validasi silang tertinggi. Dengan cara ini, pemilihan hiperparameter tidak menggunakan data uji sehingga evaluasi akhir tetap objektif.

2.7 Metrik Evaluasi Klasifikasi

Kinerja klasifikasi biner dievaluasi menggunakan *confusion matrix*, yaitu tabel 2×2 yang memuat jumlah *true positive* (TP), *true negative* (TN), *false positive* (FP), dan *false negative* (FN). Metrik yang paling umum digunakan, yaitu *accuracy*, *precision*, *recall* (sensitivitas), dan *F1-score*, dihitung dari keempat nilai tersebut (Rainio et al., 2024):

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN) \qquad \text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN) \qquad \text{F1-score} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

Akurasi menggambarkan proporsi prediksi yang benar secara keseluruhan, namun dapat menyesatkan apabila distribusi kelas tidak seimbang. Karena itu, *precision*, *recall*, dan *F1-score* per kelas turut dilaporkan. Selain itu digunakan kurva ROC (*Receiver Operating Characteristic*) yang memplot *true positive rate* terhadap *false positive rate* pada berbagai ambang probabilitas, dengan nilai AUC (*Area Under Curve*) sebagai ringkasan kemampuan model membedakan kedua kelas; AUC bernilai 0.5 berarti prediksi setara tebakan acak dan 1.0 berarti pemisahan sempurna.

BAB III

HASIL

3.1 Deskripsi Dataset dan Lingkungan Pengerjaan

Dataset yang digunakan adalah *Counter Strike 2* (berkas 03_counter_strike_2.csv) yang terdiri atas 2.000 baris dan 14 kolom. Setiap baris merepresentasikan satu *snapshot* kondisi ronde, dengan variabel target `round_winner` (CT atau T). Tidak terdapat nilai hilang pada seluruh kolom. Algoritma yang dikerjakan adalah *Naive Bayes* dan KNN. Pengolahan menggunakan bahasa Python dengan pustaka pandas, NumPy, Matplotlib, Seaborn, dan scikit-learn, dengan random_state = 42 agar hasil dapat direproduk

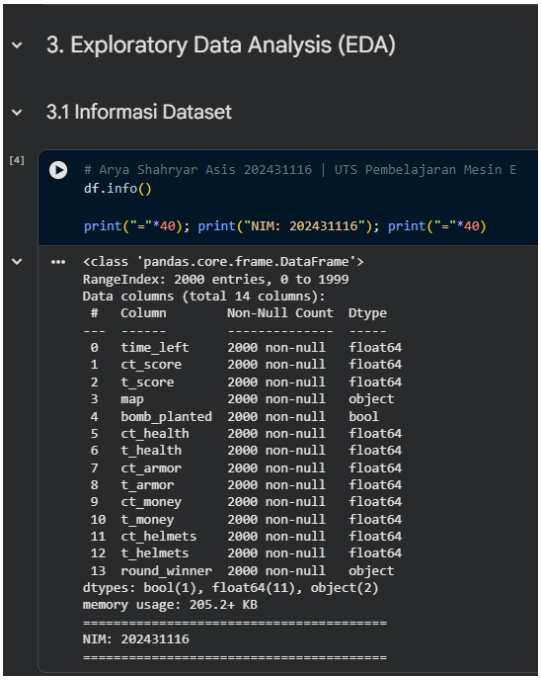
Gambar 3.1

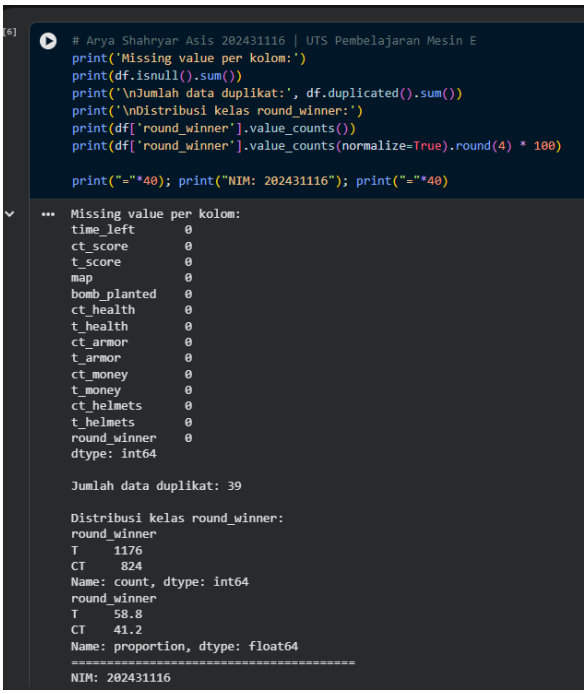


3.2 Exploratory Data Analysis (EDA)

Hasil df.info() menunjukkan seluruh 14 kolom memiliki 2.000 nilai non-null, sehingga tidak diperlukan imputasi. Terdapat 11 kolom bertipe float64, 1 kolom boolean (bomb_planted), dan 2 kolom teks (map dan round_winner). Pemeriksaan duplikat menemukan 39 baris yang identik, sehingga baris tersebut dihapus pada tahap prapemrosesan.

Gambar 3.2

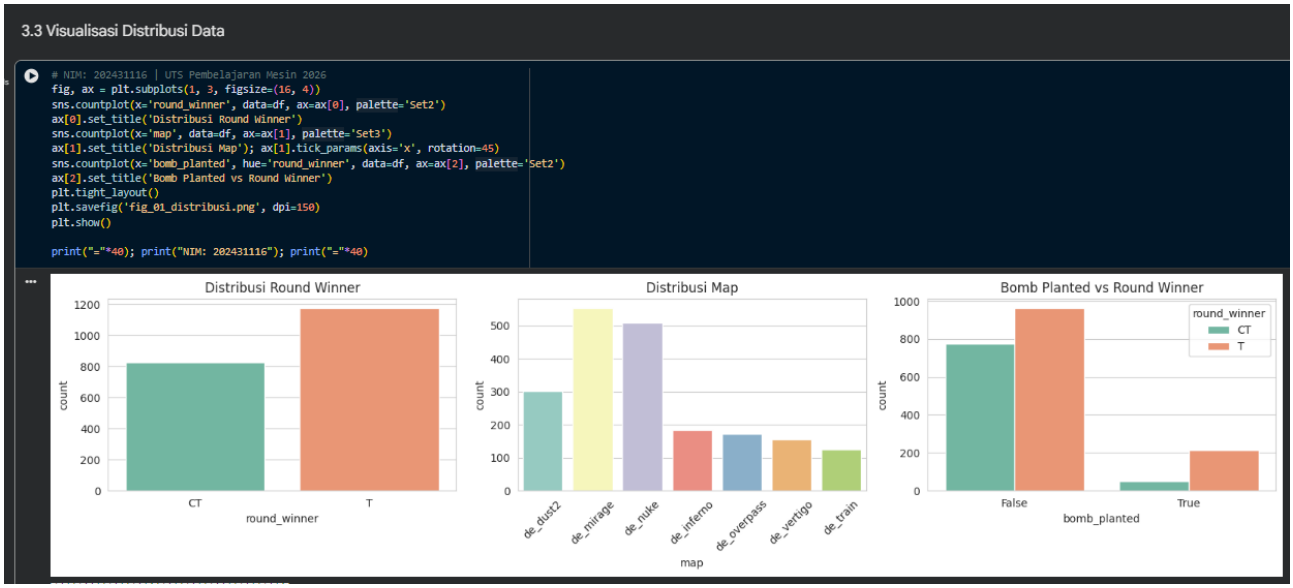




3.3 Distribusi Kelas Target

Kelas T muncul sebanyak 1.176 baris (58.8%) dan kelas CT sebanyak 824 baris (41.2%). Distribusi ini memperlihatkan ketidakseimbangan ringan dengan rasio sekitar 1.43 : 1, namun masih wajar sehingga tidak diperlukan teknik *resampling*. Konsekuensinya, akurasi tidak boleh menjadi satu-satunya acuan, dan *precision*, *recall*, serta *F1-score* per kelas perlu diperhatikan. Peta yang paling sering muncul adalah de_mirage (553 baris) dan de_nuke (509 baris), sedangkan de_train paling jarang (126 baris). Bom sudah ditanam pada 264 baris (13.2%).

Gambar 3.3

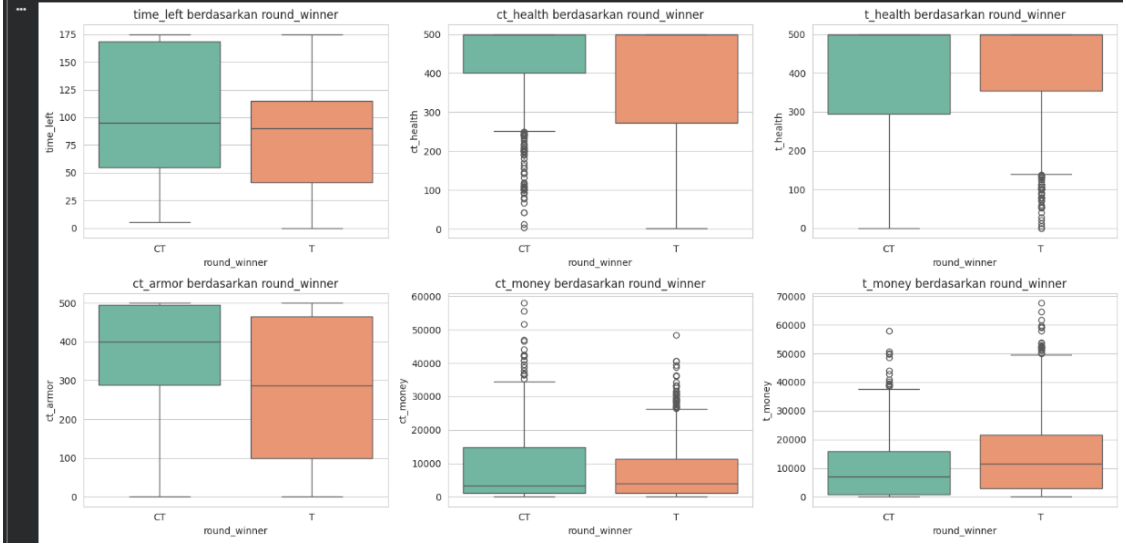




3.4 Hubungan Fitur dengan Pemenang Ronde

```
# Arya Shahryar Asis 202431116 | UTS Pembelajaran Mesin E
fig, axes = plt.subplots(2, 3, figsize=(16, 8))
for a, c in zip(axes.ravel(), ['time_left','ct_health','t_health','ct_armor','ct_money','t_money']):
    sns.boxplot(x='round_winner', y=c, data=df, ax=a, palette='Set2')
    a.set_title(f'{c} berdasarkan round_winner')
plt.tight_layout()
plt.savefig('fig_03_boxplot.png', dpi=150)
plt.show()

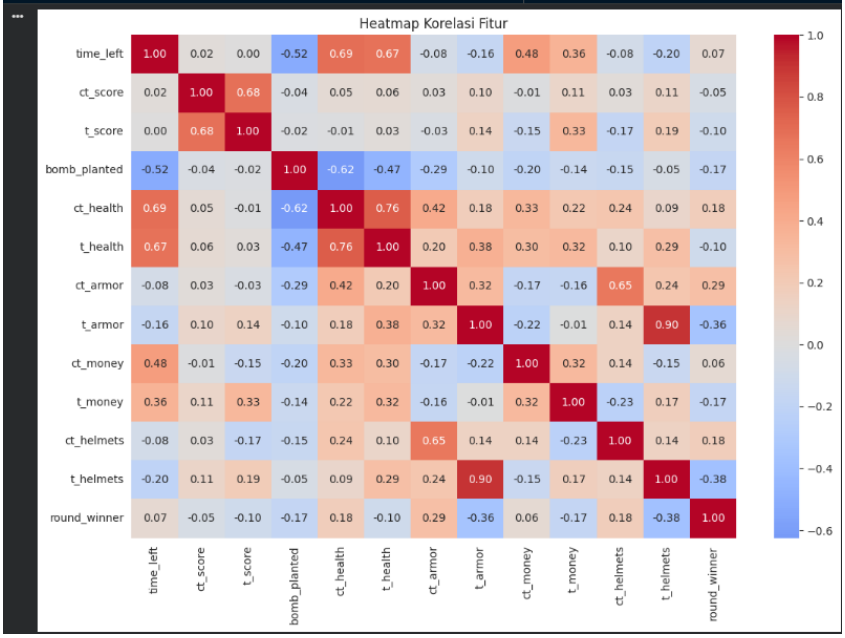
print("="*40); print("NIM: 202431116"); print("="*40)
```



Terlihat bahwa ronde yang dimenangkan CT memiliki rata-rata ct_health yang lebih tinggi (435.6 dibanding 385.5 pada ronde yang dimenangkan T) dan rata-rata t_health yang lebih rendah (386.5 dibanding 414.6). Pola serupa terlihat pada pelindung, yaitu rata-rata t_armor hanya 237.8 pada ronde yang dimenangkan CT, tetapi 359.8 pada ronde yang dimenangkan T. Rata-rata t_money juga jauh lebih tinggi pada ronde yang dimenangkan T (14.162 dibanding 9.831), sedangkan ct_money berbeda lebih tipis (7.411 pada ronde yang dimenangkan T dibanding 8.519 pada ronde yang dimenangkan CT). Dengan demikian, kondisi sumber daya tim memuat sinyal yang berguna untuk membedakan kedua kelas.

```python
# Arya Shahryar Asis 202431116 | UTS Pembelajaran Mesin E
df_corr = df.copy()
df_corr['bomb_planted'] = df_corr['bomb_planted'].astype(int)
df_corr['round_winner'] = (df_corr['round_winner'] == 'CT').astype(int)
plt.figure(figsize=(11, 8))
sns.heatmap(df_corr.drop(columns=['map']).corr(), annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Heatmap Korelasi Fitur')
plt.tight_layout()
plt.savefig('fig_04_korelasi.png', dpi=150)
plt.show()

print("="*40); print("NIM: 202431116"); print("="*40)
```



Codingan diatas matriks korelasi, dengan target dikodekan CT = 1 dan T = 0 khusus untuk analisis korelasi ini. Fitur dengan korelasi terkuat terhadap kemenangan CT adalah t_helmets (−0.376), t_armor (−0.356), dan ct_armor (+0.287), diikuti ct_health (+0.179), ct_helmets (+0.177), t_money (−0.175), dan bomb_planted (−0.173). Tanda negatif berarti semakin besar nilai fitur tersebut, semakin besar peluang tim T menang. Tidak ada satu fitur pun dengan korelasi di atas 0.4, sehingga tidak ada fitur tunggal yang cukup untuk memprediksi hasil dan diperlukan kombinasi seluruh fitur. Korelasi antarfitur dalam satu tim, misalnya ct_armor, ct_helmets, dan ct_health,

3.5 GAMBAR CODINGAN

6.3 Hyperparameter Tuning (GridSearchCV)

```
# Arya Shahryar Asis 202431116 | UTS Pembelajaran Mesin E
param_grid = {
    'n_neighbors': list(range(1, 31)),
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=cv, scoring='accuracy', n_jobs=-1)
grid.fit(X_train_sc, y_train)
print('Parameter terbaik:', grid.best_params_)
print('Akurasi CV terbaik:', round(grid.best_score_, 4))

print("="*40); print("NIM: 202431116"); print("="*40)
```

```
Parameter terbaik: {'metric': 'manhattan', 'n_neighbors': 3, 'weights': 'distance'}
Akurasi CV terbaik: 0.8814
```

6.4 Evaluasi KNN Terbaik

```
# Arya Shahryar Asis 202431116 | UTS Pembelajaran Mesin E
knn = grid.best_estimator_
y_pred_knn = knn.predict(X_test_sc)

acc_knn = accuracy_score(y_test, y_pred_knn)
print(f'Akurasi KNN (tuned): {acc_knn:.4f}')
print('\nClassification Report:')
print(classification_report(y_test, y_pred_knn, target_names=le.classes_))

ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred_knn), display_labels=le.classes_).plot(cmap='Greens')
plt.title('Confusion Matrix - KNN')
plt.savefig('fig_07_cm_knn.png', dpi=150, bbox_inches='tight')
plt.show()

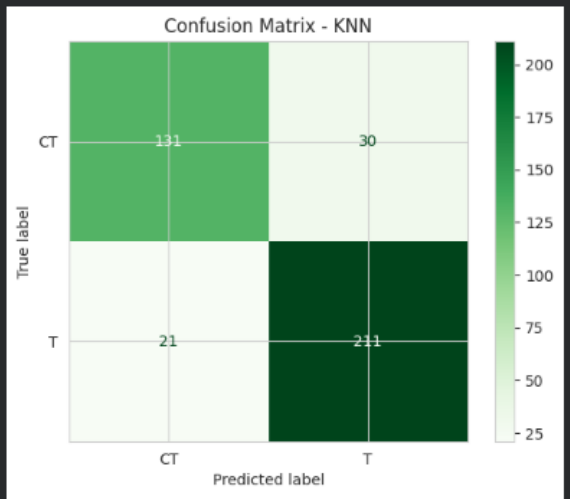
print("="*40); print("NIM: 202431116"); print("="*40)
```

```
Akurasi KNN (tuned): 0.8702

Classification Report:
              precision    recall  f1-score   support

          CT       0.86      0.81      0.84       161
           T       0.88      0.91      0.89       232

    accuracy                           0.87       393
   macro avg       0.87      0.86      0.86       393
weighted avg       0.87      0.87      0.87       393
```



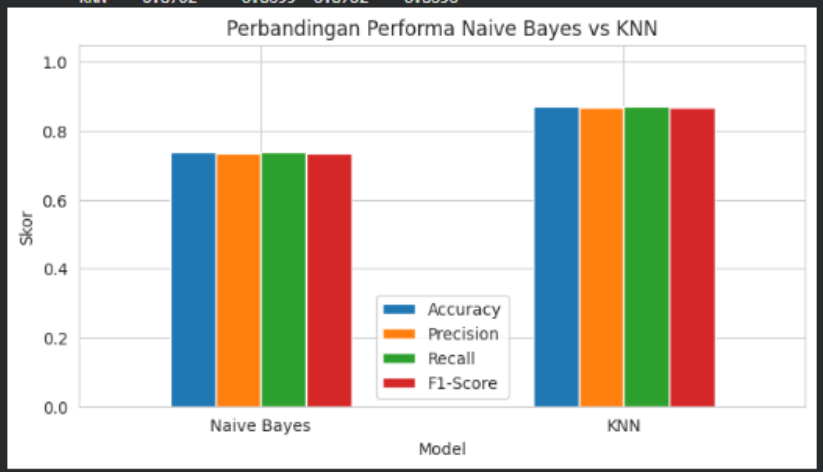
7. Perbandingan Model

```
# Arya Shahryar Asis 202431116 | UTS Pembelajaran Mesin E
hasil = pd.DataFrame({
    'Model': ['Naive Bayes', 'KNN'],
    'Accuracy':  [accuracy_score(y_test, y_pred_nb),  accuracy_score(y_test, y_pred_knn)],
    'Precision': [precision_score(y_test, y_pred_nb, average='weighted'), precision_score(y_test, y_pred_knn, average='weighted')],
    'Recall':    [recall_score(y_test, y_pred_nb, average='weighted'),    recall_score(y_test, y_pred_knn, average='weighted')],
    'F1-Score':  [f1_score(y_test, y_pred_nb, average='weighted'),        f1_score(y_test, y_pred_knn, average='weighted')],
}).round(4)
print(hasil.to_string(index=False))

hasil.set_index('Model').plot(kind='bar', figsize=(8, 4), ylim=(0, 1.05), rot=0)
plt.title('Perbandingan Performa Naive Bayes vs KNN')
plt.ylabel('Skor')
plt.savefig('fig_08_perbandingan.png', dpi=150, bbox_inches='tight')
plt.show()

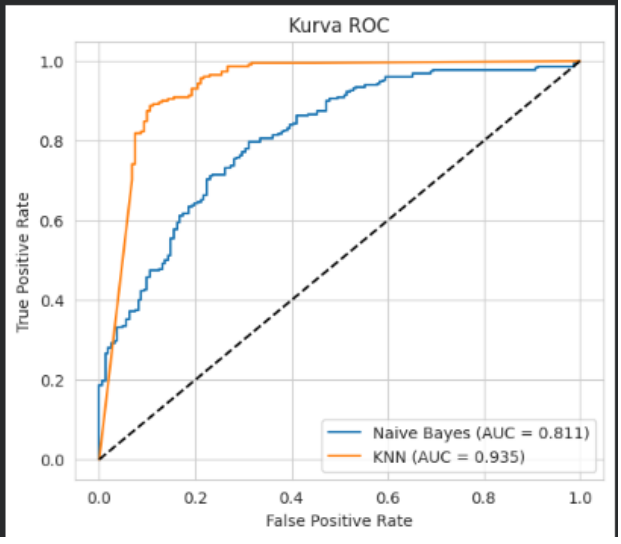
print("="*40); print("NIM: 202431116"); print("="*40)
```

```
       Model  Accuracy  Precision  Recall  F1-Score
Naive Bayes    0.7379     0.7354  0.7379    0.7349
        KNN    0.8702     0.8699  0.8702    0.8696
```




```
# Arya Shahryar Asis 202431116 | UTS Pembelajaran Mesin E
plt.figure(figsize=(6, 5))
for nama, model in [('Naive Bayes', nb), ('KNN', knn)]:
    prob = model.predict_proba(X_test_sc)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, prob)
    plt.plot(fpr, tpr, label=f'{nama} (AUC = {auc(fpr, tpr):.3f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('Kurva ROC'); plt.legend()
plt.savefig('fig_09_roc.png', dpi=150, bbox_inches='tight')
plt.show()

print("="*40); print("NIM: 202431116"); print("="*40)
```



```
# Arya Shahryar Asis 202431116 | UTS Pembelajaran Mesin E
# Cross-validation 5-fold untuk memeriksa kestabilan model
cv_nb  = cross_val_score(GaussianNB(), X_train_sc, y_train, cv=cv)
cv_knn = cross_val_score(knn, X_train_sc, y_train, cv=cv)
print(f'Naive Bayes : {cv_nb.mean():.4f} +/- {cv_nb.std():.4f}')
print(f'KNN         : {cv_knn.mean():.4f} +/- {cv_knn.std():.4f}')

print("="*40); print("NIM: 202431116"); print("="*40)
```

```
Naive Bayes : 0.7213 +/- 0.0102
KNN         : 0.8814 +/- 0.0126
========================================
NIM: 202431116
========================================
```

3.6 Penjelasan Program

Cell	Fungsi	Penjelasan
1	Import library	Mengimpor pandas dan NumPy untuk pengolahan data, Matplotlib dan Seaborn untuk visualisasi, serta modul scikit-learn untuk pembagian data, *scaling*, model (GaussianNB, KNeighborsClassifier), dan metrik evaluasi. `RANDOM_STATE = 42` ditetapkan agar hasil konsisten.
2	Load dataset	Membaca `03_counter_strike_2.csv` dengan `pd.read_csv`, mencetak ukuran data (2000, 14), dan menampilkan lima baris pertama dengan `head()`.
3	Info dataset	`df.info()` menampilkan jumlah kolom, jumlah nilai non-null, dan tipe data tiap kolom untuk memastikan tidak ada nilai hilang dan tipe data sesuai.
4	Statistik deskriptif	`df.describe().T` menghitung jumlah, rata-rata, simpangan baku, minimum, kuartil, dan maksimum tiap fitur numerik untuk mengetahui rentang dan skala fitur.
5	Missing value, duplikat, dan distribusi target	`isnull().sum()` menghitung nilai hilang, `duplicated().sum()` menghitung baris duplikat (39), dan `value_counts()` menampilkan jumlah serta persentase kelas CT dan T.
6	Visualisasi distribusi	Membuat tiga *countplot*: distribusi `round_winner`, distribusi `map`, dan `bomb_planted` berdasarkan `round_winner`. Gambar disimpan dengan `savefig`.

Cell	Fungsi	Penjelasan
7	Histogram fitur	`hist()` menampilkan distribusi 11 fitur numerik untuk mengamati bentuk sebaran, kemiringan, dan perbedaan skala.
8	Boxplot	Membuat boxplot enam fitur terhadap `round_winner` untuk membandingkan sebaran fitur pada ronde yang dimenangkan CT dan T.
9	Heatmap korelasi	Mengubah `bomb_planted` dan `round_winner` menjadi numerik, lalu menghitung korelasi Pearson antarfitur (kecuali `map`) dan menampilkannya dalam *heatmap*.
10	Hapus duplikat dan deteksi outlier	`drop_duplicates()` menghapus baris identik (2.000 → 1.961). Perulangan menghitung Q1, Q3, dan IQR tiap fitur lalu menghitung jumlah nilai di luar batas 1.5×IQR. Pencilan hanya dihitung, tidak dihapus.
11	Encoding	`bomb_planted` diubah ke 0/1, `get_dummies` melakukan *one-hot encoding* pada `map`, dan `LabelEncoder` mengubah target menjadi CT = 0 dan T = 1. Hasilnya 19 fitur.
12	Split dan standarisasi	`train_test_split` membagi data 80:20 dengan `stratify=y`. `StandardScaler` di-*fit* pada data latih dan hanya di-*transform* pada data uji agar tidak terjadi kebocoran data.
13	Model Naive Bayes	Melatih `GaussianNB`, memprediksi data uji, menghitung akurasi (0.7379), mencetak `classification_report`, dan menampilkan *confusion matrix*.
14	KNN baseline	Melatih `KNeighborsClassifier(n_neighbors=5)` dan menghitung akurasinya (0.7990) sebagai pembanding sebelum penyetelan.
15	Pencarian nilai k	Menghitung rata-rata akurasi validasi silang `StratifiedKFold` 5-*fold* untuk k = 1–30, menentukan k terbaik dengan `argmax`, dan memplot akurasi terhadap k.

Cell	Fungsi	Penjelasan
16	GridSearchCV	Mendefinisikan `param_grid` (k, `weights`, `metric`) dan menjalankan `GridSearchCV` dengan validasi silang 5-*fold*. Hasil: k = 3, `distance`, `manhattan`, akurasi CV 0.8814.
17	Evaluasi KNN terbaik	Mengambil `best_estimator_`, memprediksi data uji, menghitung akurasi (0.8702), mencetak `classification_report`, dan menampilkan *confusion matrix*.
18	Perbandingan model	Menghitung *accuracy*, *precision*, *recall*, dan *F1-score* (rata-rata tertimbang) kedua model dalam satu tabel, lalu memvisualisasikannya dengan diagram batang.
19	Kurva ROC	Mengambil probabilitas kelas T dari `predict_proba`, menghitung `roc_curve` dan AUC kedua model (0.811 dan 0.935), lalu memplot keduanya.
20	Cross-validation akhir	`cross_val_score` 5-*fold* pada kedua model untuk memeriksa kestabilan: *Naive Bayes* 0.7213 ± 0.0102 dan KNN 0.8814 ± 0.0126.

BAB IV
PENUTUP

4.1 Kesimpulan

1. Dataset *Counter Strike 2* terdiri atas 2.000 baris dan 14 kolom tanpa nilai hilang, dengan 39 baris duplikat. Distribusi kelas target cukup wajar, yaitu 58.8% kelas T dan 41.2% kelas CT, sehingga tidak memerlukan *resampling* tetapi mengharuskan evaluasi dengan metrik per kelas. Fitur uang berskala jauh lebih besar dan berdistribusi miring ke kanan dibanding fitur lain. Fitur dengan hubungan terkuat terhadap hasil ronde adalah t_helmets, t_armor, dan ct_armor, dan ronde dengan bom yang sudah ditanam lebih sering dimenangkan tim T (80.7%).
2. Prapemrosesan yang dilakukan meliputi penghapusan duplikat (menjadi 1.961 baris), analisis *outlier* dengan metode IQR (dipertahankan karena merupakan kondisi permainan yang valid), *encoding* menjadi 19 fitur, pembagian data berstrata 80:20, dan standarisasi dengan StandardScaler yang hanya dilatih pada data latih. Standarisasi diperlukan terutama untuk KNN yang berbasis jarak.
3. Algoritma *Naive Bayes* menghasilkan akurasi 0.7379 dengan AUC 0.811. Kinerjanya lebih rendah pada kelas CT (*recall* 0.62). Hal ini sejalan dengan teori, karena asumsi independensi bersyarat tidak terpenuhi akibat fitur-fitur dalam satu tim saling berkorelasi, dan asumsi distribusi normal tidak sesuai dengan fitur uang yang miring.
4. Algoritma KNN dengan hiperparameter terbaik hasil *Grid Search* (k = 3, bobot *distance*, jarak *Manhattan*) menghasilkan akurasi 0.8702 dengan AUC 0.935. Kinerja kedua kelas relatif seimbang (*F1-score* 0.84 untuk CT dan 0.89 untuk T). Penyetelan hiperparameter terbukti meningkatkan kinerja, dari 0.7990 pada konfigurasi bawaan (k = 5) menjadi 0.8702.
5. KNN lebih baik daripada *Naive Bayes* pada dataset ini dengan selisih akurasi 13.23 poin persentase. KNN tidak mengasumsikan bentuk distribusi maupun independensi fitur sehingga mampu menangkap hubungan antarfitur yang kompleks. Namun, hasil ini perlu ditafsirkan hati-hati karena dataset berupa *snapshot* tanpa identitas ronde, sehingga ada kemungkinan *snapshot* dari ronde yang sama berada pada data latih dan data uji sekaligus, yang menguntungkan KNN.

DAFTAR PUSTAKA

1. Halder, R. K., Uddin, M. N., Uddin, M. A., Aryal, S., & Khraisat, A. (2024). Enhancing K-nearest neighbor algorithm: A comprehensive review and performance analysis of modifications. *Journal of Big Data, 11*, 113. https://doi.org/10.1186/s40537-024-00973-y
2. Hasegawa, K., & Hirota, M. (2024). A study on predicting the winning team of a round in Counter-Strike: Global Offensive using frame information. *Proceedings of Summer Conference, Digital Game Research Association Japan*, 47–51.
3. Liu, X.-Q., Wang, X.-C., Tao, L., An, F.-X., & Jiang, G.-R. (2024). Alleviating conditional independence assumption of naive Bayes. *Statistical Papers, 65*(5), 2835–2863. https://doi.org/10.1007/s00362-023-01474-5
4. Nayak, S., Bhat, M., Reddy, N. V. S., & Rao, B. A. (2022). Study of distance metrics on k-nearest neighbor algorithm for star categorization. *Journal of Physics: Conference Series, 2161*(1), 012004. https://doi.org/10.1088/1742-6596/2161/1/012004
5. Pagan, M., Zarlis, M., & Candra, A. (2023). Investigating the impact of data scaling on the k-nearest neighbor algorithm. *Computer Science and Information Technologies, 4*(2), 135–142. https://doi.org/10.11591/csit.v4i2.p135-142
6. Pinheiro, J. M. H., et al. (2025). The impact of feature scaling in machine learning: Effects on regression and classification tasks. *IEEE Access*. https://doi.org/10.1109/ACCESS.2025.3635541
7. Rainio, O., Teuho, J., & Klén, R. (2024). Evaluation metrics and statistical tests for machine learning. *Scientific Reports, 14*, 6086. https://doi.org/10.1038/s41598-024-56706-x
8. Sinap, V. (2023). Makine öğrenmesi teknikleri ile Counter-Strike: Global Offensive raunt sonuçlarının tahminlenmesi. *Journal of Intelligent Systems: Theory and Applications, 6*(2), 119–129.
9. Xenopoulos, P., Coelho, B., & Silva, C. (2021). Optimal team economic decisions in Counter-Strike. *IJCAI 2021 Workshop on Artificial Intelligence for Sports Analytics (AISA)*. arXiv:2109.12990.
10. Zhang, S. (2022). Challenges in KNN classification. *IEEE Transactions on Knowledge and Data Engineering, 34*(10), 4663–4675.